

KRCIGE：一个有界工程框架

从认识、资源、控制与工程不可逆性推导软件工程原则

作者信息待补

单位与电子邮箱待补

理论化修订 v0.2

2026 年 9 月 6 日

摘要

软件工程长期积累了大量经验原则，例如 DRY、低耦合、小步迭代、版本控制、自动化测试、显式契约、最少共享状态与持续反馈。本文将 KRCIGE 表述为一个面向软件工程行动的情境化序贯决策框架，用统一的任务、主体、时间窗和策略模型组织这些原则。

框架包含四个描述性要素：认识有限（K）、资源有限（R）、控制有限（C）与工程不可逆性（I）；两个规范性要素：工程目标（G）与伦理边界（E）；以及可学习结构条件 Σ 。K、R、C 的强度分别由决策相关不确定性、有效资源瓶颈和控制缺口表示，I 表示恢复到任务等价状态所需的最低预期成本。本文给出统一形式模型、十三条可检验的中间原则、序贯发布案例、I 的区分性命题、跨材料验证设计，并以 *The Pragmatic Programmer* 100 条 Tips 和 Twelve-Factor App 展示框架的覆盖与前瞻性使用。

本文的核心主张是：在情境 S_Q 下，K、R、C、I 通过目标函数、可行策略集合和恢复约束共同改变策略的相对价值；P1-P13 提供从约束到工程实践的可检验机制层。框架保留各量的原有量纲，通过价值函数、约束集合和 Pareto 比较完成跨维度决策。

关键词：软件工程理论；有限理性；序贯决策；工程不可逆性；恢复成本；可检验性

1 引言

软件工程拥有数量庞大的经验原则。它们来自不同年代、不同语言、不同组织规模 and 不同技术栈，却经常指向相似的工程行为：减少无关耦合、缩小一次变更的影响范围、保留历史、自动化重复工作、尽快取得反馈、显式表达假设、控制共享状态、让架构保持可修改性。

一个自然的问题是：这些经验是否共享更少、更基础的原因？

本文提出 KRCIGE 框架，试图把软件工程原则组织为以下推导结构：

$$\boxed{\text{现实约束} + \text{目标与价值边界} \rightsquigarrow \text{中间工程原则} \rightsquigarrow \text{具体实践}} \quad (1.1)$$

KRCIGE 的目标包含三个层次：

- (1) **压缩性：**用较少基础命题解释较多经验原则；
- (2) **生成性：**从基础命题推出新的工程判断；
- (3) **条件性：**解释某条经验原则在什么条件下增强、减弱或需要与另一条原则共同优化。

本文的理论对象定义为工程情境 S_Q 及其可行策略集合。给定情境后，框架输出策略价值、恢复要求和可观测预测；每条中间原则对应机制、指标和适用边界。理论的主要增量集中在任务相关的工程不可逆性 $I_{\varepsilon, \delta}$ 、决策相关认识缺口 k_Q 、资源影子价格 λ_j 和控制缺口 γ_Q 的联合使用。

该框架与若干已有理论传统具有明显联系。Simon 的有限理性强调知识、记忆与计算能力的约束 [1]; Ashby 的必要多样性定律讨论控制器能力与环境扰动之间的关系 [3]; Landauer 研究逻辑不可逆计算与信息擦除的物理意义 [4]; Parnas 将模块化与可理解性、可修改性联系起来 [5]; SICP 将数据抽象解释为最少承诺原则的一个应用 [6]; FLP 结果展示了模型假设对分布式系统可实现保证的约束 [7]; No Free Lunch 结果提醒我们, 算法优势依赖问题类别所提供的结构 [8]。

本文将这些思想组织为一个面向软件工程实践的统一框架。

2 KRCIGE 理论框架与基本模型

定义 2.1 (工程情境). 工程情境记为

$$\mathcal{S}_Q = (Q, s, \tau, O, \mathcal{A}_s, T, B, \mathcal{A}_E).$$

任务 Q 给出状态空间 $\mathcal{W}$ 、目标价值、损失度量和恢复要求; 主体 s 给出控制权限与观测接口; 有限时间窗 $\tau = \{0, \dots, H\}$ 给出交付、反馈与恢复的评价范围。 B 为初始资源向量, $\mathcal{A}_s$ 为主体权限下的行动集合, $\mathcal{A}_E(h_t) \subseteq \mathcal{A}_s$ 为历史 h_t 下伦理允许的行動集合。

状态、观测与行动满足

$$w_{t+1} = T(w_t, a_t, d_t), \quad o_t = O(w_t), \quad h_t = (o_0, a_0, \dots, a_{t-1}, o_t), \quad a_t = \pi(h_t). \quad (2.1)$$

带噪观测由观测核 $P(o_t | w_t)$ 表示; d_t 为外部扰动, $\mu_t = P(w_t | h_t)$ 为当前信念。策略 π 在每个时刻依据已经获得的历史选择行动。定义 $c_t \geq 0$ 为资源消耗向量, $b_t = B - \sum_{\ell < t} c_\ell$ 为剩余预算, 并定义

$$\Pi_Q(h_t, b_t) = \left\{ \pi : a_\ell = \pi(h_\ell) \in \mathcal{A}_E(h_\ell), \sum_{\ell=t}^{H-1} c_\ell \preceq b_t \text{ 几乎处处} \right\}. \quad (2.2)$$

根节点记为 $\Pi_Q(B) = \Pi_Q(h_0, B)$, 其中 $h_0 = (o_0)$ 。预算具有逐分量单位, 可包含时间、算力、存储、金钱、注意力和协作工时。概率预算和期望预算可以作为情境中另行登记的约束形式。

定义 2.2 (轨迹价值与可行决策). 令 $\omega = (w_0, a_0, \dots, w_H)$ 为轨迹, v_t 为交付价值, ℓ_t 为直接后果损失, κ_t 为行动资源消耗的价值折算, v_H 为终止价值。定义

$$U_Q(\omega) = \sum_{t=0}^{H-1} \zeta^t (v_t - \ell_t - \kappa_t) + \zeta^H v_H, \quad 0 < \zeta \leq 1, \quad (2.3)$$

$$V_Q(h_t, b_t) = \sup_{\pi \in \Pi_Q(h_t, b_t)} \mathbb{E}[U_{Q,t:H}(\pi) | h_t]. \quad (2.4)$$

$\Pi_Q(h_t, b_t)$ 表示从当前历史继续执行的可行策略。最优值由上确界定义; 最优策略存在时, π_Q^* 取达到该值的策略。行动集合、效用和分布固定后, 有限模型可由后向递推求解。

ℓ_t 记录用户、业务和系统直接后果; 恢复操作占用的工时、计算和协调资源进入 κ_t 。每项代价在价值账本中归属一个条目。资源预算同时表达物理可行性, 价值折算表达资源使用的机会代价。伦理条件通过可行集合表达; 多目标情境采用预先约定的偏好序或 Pareto 前沿。

K、R、C、I 是同一情境的诊断量。 k_Q 的单位为目标价值; B_j 的单位为资源 j , λ_j 的单位为价值/资源 j ; γ_Q 的单位为目标偏离损失; $I_{\epsilon, \delta}$ 的单位为约定的恢复成本。跨项目比较固定时

间窗、价值尺度和成本折算，或公开尺度转换与测量误差。恢复成本经价值折算进入 U_Q ，恢复质量与可靠性进入可行条件。

3 推导语义

形式蕴含。 $A \implies B$ 表示在给定定义和数学前提下的严格逻辑后果。命题随文登记可行域、概率模型与量纲，并提供证明。

条件性工程压力。 $A \rightsquigarrow P_i$ 表示情境条件 A 通过某个机制提高实践 P_i 相对于指定替代策略的预期价值。定义实践及替代策略的净价值差为

$$\Delta V_i(\theta) = V_Q(\pi_i; \theta) - V_Q(\pi_0; \theta).$$

θ 表示已登记的情境参数。每条方向假设规定参数变化方式、比较对象、保持固定的量和 ΔV_i 的符号或变化方向。 $\Delta V_i = 0$ 给出策略选择边界。

要素合取与交互。 路径 $K + R + C \rightsquigarrow P_i$ 中的加号表示同一情境下条件的合取。数值交互由 $\Delta V_i(\theta)$ 中的函数关系表达。例如在成本模型规定防护减少固定比例 ξ 的恢复负担时，其收益包含 $p\xi I$ ；交互的形式和单位随模型一起登记。

经验桥接。 辅助前提 Ψ_j 的记录为

$$\Psi_j = (\text{具体主张, 适用人群与任务, 证据来源, 效应区间, 更新日期}).$$

涉及心理、组织与工具效果的路径引用对应记录。 Ψ_j 作为模型参数和解释复杂度的一部分进入检验。本文的十三条原则按条件性工程压力解释，后续研究以独立材料估计桥接效应。

4 相关理论定位

KRCIGE 与已有理论形成层次互补关系：已有理论提供一般机制，KRCIGE 将这些机制放入工程情境 $\mathcal{S}_Q$ ，并通过策略价值、资源预算、控制缺口与恢复成本连接到软件实践。

4.1 有限理性与价值信息

Simon 的有限理性为 K 提供认知约束背景，价值信息分析为 P1 提供决策价值表达 [1]。KRCIGE 使用 k_Q 表示任务相关的期望信息价值，并将观测成本、策略集合和伦理边界同时纳入比较。

4.2 实物期权与工程不可逆性

实物期权理论把不可逆投资、等待和未来选择空间放入同一价值模型 [13]。KRCIGE 的 P2 沿用选择权机制，I 进一步以 $I_{\epsilon, \delta}$ 表示软件行动恢复到任务等价状态的最低预期成本；版本历史、feature flag、备份和追加式发布账本因此成为可计算的选择权与证据结构。

4.3 控制论与弹性工程

Ashby 的必要多样性定律、FLP 的模型边界和鲁棒控制共同说明控制能力取决于扰动集合、信息和策略权限 [3, 7, 14]。KRCIGE 以 γ_Q 将这种关系写成任务损失，并以 P4、P9、P10 和 P11 连接到局部化、状态空间、恢复和安全实践。弹性工程强调系统吸收扰动、持续运行和恢复能力 [15]；KRCIGE 以恢复证据、恢复成功率和 MTTR 提供对应的工程观测接口。

4.4 模块化、抽象与技术债务

Parnas 的模块划分和 SICP 的最少承诺原则分别对应 P4 与 P2 [5, 6]。技术债务可以在 KRCIGE 中表示为延后决策或结构承诺带来的未来变更成本：

$$C_{\text{debt}} = C_{\text{deferred}} + C_{\text{interest}}.$$

P3 记录实现、理解、运行和变更成本，P13 依据新证据更新债务估算；该表示把设计取舍、维护负担和未来选择权放入同一个成本模型。

5 理论要素与适用条件

5.1 K: 认识有限

定义 5.1 (决策相关认识缺口). 固定任务 Q 、主体 s 、时间窗 τ 、当前观测历史 h_t 和剩余预算 B 。令 $\mathcal{A}_Q(B)$ 为伦理与预算允许的候选决策集合， $u_Q(w, a)$ 为候选决策的条件期望净价值，其中未来扰动按登记的条件分布积分。定义

$$k_Q(s, \tau, h_t; B) = \mathbb{E}_{W|h_t} \left[\sup_{a \in \mathcal{A}_Q(B)} u_Q(W, a) \right] - \sup_{a \in \mathcal{A}_Q(B)} \mathbb{E}_{W|h_t} [u_Q(W, a)]. \quad (5.1)$$

该式采用相同的候选集合和价值尺度。 k_Q 表示完整相关状态信息的期望决策价值， $k_Q > 0$ 表示存在会影响行动选择的认识有限性 K。

W 在此表示决策前的任务相关状态或参数，例如兼容性参数、需求类型和依赖版本。 k_Q 给出当前模型内信息价值的基准，观测收益由实际可获得的信号及其成本计算。单阶段情境中的 a 表示行动；序贯情境中的候选决策包含已登记的继续执行方案。Simon 的有限理性提供认知约束背景，统计决策中的信息价值提供量化接口 [1, 2]。

5.2 R: 资源有限

定义 5.2 (资源瓶颈). 给定当前历史 h_t ，资源有限性 R 由预算向量 $B = (B_1, \dots, B_m)$ 、行动成本 $c(a)$ 和可行域

$$\mathcal{A}_Q(B) = \{a \in \mathcal{A}_E(h_t) : c(a) \preceq B\}$$

表示。固定历史和其他情境参数，以第 2 节的 $V_Q(h_t, B)$ 为预算价值函数，定义资源 j 在预算增量 $\Delta > 0$ 下的边际价值

$$\lambda_j(\Delta) = \frac{V_Q(h_t, B + \Delta e_j) - V_Q(h_t, B)}{\Delta}. \quad (5.2)$$

e_j 为第 j 个单位向量。 $\lambda_j(\Delta) > 0$ 表示资源 j 在该增量尺度下构成有效瓶颈；可微情境中的局部影子价格为 $\lambda_j = \partial V_Q / \partial B_j$ 。

离散预算登记增量 Δ ，连续预算登记导数存在的范围。资源替代通过扩大一组预算、调整另一组资源消耗并重新求值来评价。资源消耗、可用预算和价值尺度共同决定瓶颈强度；恢复能力通过恢复工时、备用容量和协调权限等具体投入表示。

5.3 C: 控制有限

定义 5.3 (控制缺口). 固定当前信念 $P(W | h_t)$ 、扰动序列集合 $\mathcal{D}_\tau$ 、目标集合 $\mathcal{G}_Q$ 和非负目标偏离损失 L_Q 。 T_τ 表示时间窗内策略执行的结果，定义

$$\gamma_Q(s, \tau, h_t; B) = \inf_{\pi \in \Pi_Q(h_t, B)} \sup_{d \in \mathcal{D}_\tau} \mathbb{E}_{W|h_t} [L_Q(T_\tau(W, \pi, d), \mathcal{G}_Q)]. \quad (5.3)$$

γ_Q 为可行策略应对所登记扰动集合的最小最坏情形期望损失。 $\gamma_Q > 0$ 表示目标上的控制有限性 C。

该控制指标采用鲁棒评价；第 2 节的策略价值采用已登记的概率评价。情境可以同时报告二者，并将 $\gamma_Q \leq g_{\max}$ 作为保证要求。扰动集合可以包含网络、调度、第三方接口与外部行为；反馈信息和执行权限决定策略所能利用的行动。比较控制权限时固定信念与预算，比较认识状态时固定权限与扰动模型，由此识别各因素的条件效应 [3, 14]。

5.4 I: 工程不可逆性

工程行动的恢复要求由任务状态差异、证据保留和伦理允许的恢复策略共同确定。令 $X \sim P(\cdot | h_t)$ 为行动前状态， $Y = T(X, a, D)$ 为行动后状态， z 为行动前后持续可用的证据， $\rho \in \Pi_E$ 为满足伦理边界的恢复策略，恢复状态记为 X^ρ 。定义任务差异度量 $D_Q(x, x') \geq 0$ ，允许差异为 $\varepsilon \geq 0$ ，最大失败概率为 $\delta \in [0, 1]$ 。

定义 5.4 (工程不可逆性). 行动 a 在情境 $(Q, h_t, z, \varepsilon, \delta)$ 下的工程不可逆性为

$$I_{\varepsilon, \delta}(a | h_t, z) = \inf_{\rho \in \Pi_E} \{\mathbb{E}[C_{\text{rec}}(\rho; Y, z) | h_t, z] : \Pr[D_Q(X, X^\rho) \leq \varepsilon | h_t, z] \geq 1 - \delta\}. \quad (5.4)$$

若满足条件的恢复策略集合为空，则规定 $I_{\varepsilon, \delta} = +\infty$ 。 C_{rec} 可以保留为资源向量；标量表达采用事先登记的价值折算。若最优策略存在， I 是其最低预期恢复成本；一般情境使用已知策略的可实现上界并记录策略集合。

$I_{\varepsilon, \delta}$ 衡量行动后的恢复负担， k_Q 衡量行动前的信息价值；二者由同一状态模型连接，并保持不同逻辑角色。工程目标 G 决定 D_Q, ε, δ ，伦理边界 E 决定 Π_E ，资源约束决定恢复策略的可行性。

命题 5.5 (信息保留的恢复单调性). 设 $z_2 = (z_1, \tilde{z})$ 是 z_1 的信息细化，且在比较中固定条件状态分布、恢复成本和策略集合。则

$$I_{\varepsilon, \delta}(a | h_t, z_2) \leq I_{\varepsilon, \delta}(a | h_t, z_1).$$

若 z_2 使行动前的任务相关状态在后验上可区分，而 z_1 使其中至少两个正概率状态不可区分，则在精确恢复要求 $(\varepsilon, \delta) = (0, 0)$ 下， $I_{0,0}(a | h_t, z_1) = +\infty$ 。

证明：在固定条件状态分布的比较中，使用 z_2 的恢复策略可以先忽略新增信息并复现任一使用 z_1 的策略，因此可行策略集合包含关系给出第一式。第二式来自确定性恢复无法从同一 (Y, z_1) 区分两个任务相关状态。 $\square$

命题 5.6 (不可逆性对策略排序的贡献). 考虑两个行动 a_1, a_2 ，其成功价值、缺陷概率和直接缺陷损失相同，恢复成本的可行最优值分别为 $I_1 < I_2$ ，行动成本分别为 C_1, C_2 ，缺陷概率为 $p > 0$ 。在相同价值尺度下，

$$\mathbb{E}[U(a_1) - U(a_2)] = p(I_2 - I_1) - (C_1 - C_2).$$

因此当 $p(I_2 - I_1) > C_1 - C_2$ 时， a_1 的期望效用更高。该排序隔离了恢复路径和证据结构对策略价值的贡献。

信息压缩 $f: \mathcal{X} \rightarrow \mathcal{Y}$ 可产生结构性不可逆性。若存在正后验概率的 $x_1 \neq x_2$ 满足 $f(x_1) = f(x_2)$ ，且 z 不区分二者，则精确恢复要求下 $I_{0,0} = +\infty$ 。保存原始值、版本历史、日志和备份扩充证据 z ，从而降低可实现恢复成本。例如时区、精度和错误类型被压缩掉后，任务相关状态会落入同一表示；显式表示保留这些语义。

5.5 G: 工程目标

定义 5.7 (工程目标). 工程目标 G 给出对系统状态和行动的偏好。本文采用宽泛表述：

G : 在约束下持续、可靠地交付预期价值。

G 提供规范方向。 K, R, C, I 描述工程世界的约束， G 决定工程主体希望在这些约束中优化什么。

5.6 E: 伦理边界

定义 5.8 (伦理边界). 伦理边界 E 为每个决策历史定义允许行动集合

$$\mathcal{A}_E(h_t) \subseteq \mathcal{A}_s.$$

工程策略的行动满足

$$a_t \in \mathcal{A}_E(h_t).$$

这一层用于表达安全、隐私、公平、责任和社会伤害等约束。工程目标可以具有多个量化指标，伦理边界为部分行动提供硬约束或高优先级约束。

5.7 Σ : 可学习结构条件

定义 5.9 (可学习结构条件). 给定参考环境集合 $\mathcal{E}_Q$ 、观测 Z 、当前历史 h_t 和成本模型，定义观测的净决策价值

$$\text{NVOI}_{Q,e}(Z | h_t) = \mathbb{E}_{Z|h_t,e} \left[\sup_{a \in \mathcal{A}_Q(B-c_Z)} \mathbb{E}[U_Q | Z, h_t, e, a] \right] - \sup_{a \in \mathcal{A}_Q(B)} \mathbb{E}[U_Q | h_t, e, a] - C_Q(Z). \quad (5.5)$$

当存在 $\eta \in [0, 1)$ 使

$$\Pr_{e \sim \mathcal{E}_Q} [\text{NVOI}_{Q,e}(Z | h_t) > 0] \geq 1 - \eta$$

时, 称 $(Z, \mathcal{E}_Q)$ 满足可学习结构条件 Σ 。 η 表示参考环境中净决策价值为正的比例缺口; 环境集合、观测窗口和成本模型随研究预注册。

互信息

$$\mathcal{I}_{\text{Shannon}}(Z; Y) > 0$$

提供结构性关联指标, 其中 Y 为未来关注结果。净决策价值进一步要求关联能改变可行行动的期望净价值。No Free Lunch 结果说明算法优势依赖问题类别中的结构假设 [8]。 K 、 R 、 C 描述能力和预算边界, I 描述行动后的恢复边界, G 、 E 提供规范方向, Σ 限定反馈能够积累策略价值的环境。

6 中间原则

为减少从 KRCIGE 到具体实践的重复推导, 本文定义十三条中间原则。

6.1 P1: 反馈与信息价值

派生原则 6.1 (反馈价值原则). 在 $K + R + \Sigma + G$ 条件下, 当一项观测的预期决策价值高于获取成本时, 应优先获得该观测。

在统一模型中, 观测的净决策价值由式(5.5) 给出。当

$$\text{NVOI}_{Q,e}(Z \mid h_t) > 0$$

时, 观测具有正工程价值。

由此可导出原型、示踪子弹、自动测试、性能测量、用户反馈、端到端验证和迭代需求发现。

6.2 P2: 选择权与最少承诺

派生原则 6.2 (选择权原则). 在 $K + I + R + G$ 条件下, 未来不确定性越高、候选行动的工程不可逆性越强, 保留选择权的价值越高。

可以粗略定义等待的期权价值:

$$V_{\text{wait}} = V_{\text{future choice}} - C_{\text{delay}}.$$

当等待带来的未来选择价值覆盖延迟成本时, 推迟不可逆承诺具有正价值。

SICP 将数据抽象解释为最少承诺原则的应用: 抽象屏障允许具体表示选择延后, 从而保留系统灵活性 [6]。

6.3 P3: 复杂度预算

派生原则 6.3 (复杂度预算原则). 复杂度消耗有限资源。新增复杂度应带来足以覆盖实现、理解、运行和变更成本的工程价值。

可以使用总成本表达:

$$C_{\text{total}} = C_{\text{implementation}} + C_{\text{understanding}} + C_{\text{operation}} + C_{\text{change}}.$$

该原则支持简单设计、渐近复杂度分析、减少无收益抽象、控制继承层次和避免无价值技术迁移。

6.4 P4: 局部化与模块化

派生原则 6.4 (局部化原则). 在 $K + R + C + G$ 条件下, 应缩小一次变更要求同时理解、协调和控制的范围。

可以把模块化目标写成

$$\min (\mathbb{E}[C_{\text{propagation}}] + C_{\text{boundary}}).$$

第一项是变化传播成本, 第二项是接口、部署、通信和边界管理成本。

Parnas 的经典工作将模块划分与灵活性、可理解性联系起来 [5]。该表达同时允许解释单体模块化、服务边界和微服务粒度的权衡。

6.5 P5: 显式假设

派生原则 6.5 (显式假设原则). 在 $K + C + I$ 条件下, 跨越模块、团队或时间边界的重要假设, 应尽量编码为显式、可验证的数据、类型、契约或测试。

该原则产生 Design by Contract、断言、schema、领域类型、显式状态、属性测试与可执行规范。

6.6 P6: 信息保留与证据价值

派生原则 6.6 (信息保留原则). 在满足伦理边界的前提下, 若某项信息未来被需要的概率为 p , 丢失后增加的恢复或决策损失为 L , 保存成本为 S , 保存带来的安全与隐私风险为 Q , 则在

$$pL > S + Q$$

时保存具有正期望价值。

该原则支持版本历史、结构化日志、审计轨迹、原始事件、诊断上下文和高精度中间表示。被伦理边界禁止的保存行为首先从可行集合中排除, 其余方案再比较未来决策价值、恢复负担、存储成本与风险。

6.7 P7: 权威来源与协调语义

派生原则 6.7 (权威来源原则). 当同一事实存在多个副本时, 应显式给出权威来源或冲突协调语义。

其推导来自

$$R + C + I + G.$$

多副本同步消耗资源; 外部更新和并发变化增加分叉概率; 分叉会削弱“哪个副本表达当前事实”的信息。

这一原则给出 DRY 的一个更一般版本:

同一权威知识需要明确 authority 或 reconciliation。

6.8 P8: 自动化阈值

派生原则 6.8 (自动化原则). 设人工流程执行 n 次的预期成本为

$$nC_h + R_h,$$

自动化方案成本为

$$F + nC_a + R_a.$$

当

$$F + nC_a + R_a < nC_h + R_h$$

时, 自动化具有正期望收益。

这里 F 为自动化固定成本, C_h, C_a 为每次执行成本, R_h, R_a 为失败与变异风险成本。

6.9 P9: 状态空间控制

派生原则 6.9 (状态空间原则). 在 $K + R + C$ 条件下, 工程设计应控制需要推理、测试和协调的可达状态空间。

共享可变状态会增加可能 interleaving 的数量。消息传递、不可变数据、事务边界、actor 和状态机都可以作为状态空间约束技术。

该原则允许受控共享状态。数据库事务、锁、共识协议和明确的一致性模型都可以提供协调语义。

6.10 P10: 不可逆性与恢复能力

派生原则 6.10 (可恢复性原则). 行动的工程不可逆性越高, 预防错误和降低恢复负担的机制通常具有越高价值。

设防护机制的成本为 C_{guard} , 行动发生错误的概率为 p_{error} , 防护机制能够减少的错误后损失为 $\Delta L(I_{\epsilon, \delta})$ 。当

$$C_{\text{guard}} < p_{\text{error}} \Delta L(I_{\epsilon, \delta})$$

时, 防护机制具有正期望价值。在其他条件相近时, $I_{\epsilon, \delta}$ 上升会提高 review、dry-run、备份、分阶段发布、幂等、授权和审计等机制的价值。

数据库迁移、支付、权限变更和数据删除都可以依据该原则配置不同强度的验证与恢复机制。

6.11 P11: 安全暴露面与爆炸半径

派生原则 6.11 (安全约束原则). 在 $R + C + I + E$ 条件下, 应降低攻击者可操纵的状态空间, 并限制单次失效能够造成的最大伤害。

这产生最小权限、较小攻击面、快速安全修补、隔离和数据最小化等实践。

6.12 P12: 知识外部化

派生原则 6.12 (知识外部化原则). 重要工程知识应具有超越单个人脑的持久表达。

推导路径为

$$\begin{aligned} K + R + I + G &\rightsquigarrow \text{个人知识容量有限且会随时间流失,} \\ &\rightsquigarrow \text{文档、代码、测试、术语表、评审与版本历史.} \end{aligned}$$

6.13 P13: 模型持续校正

派生原则 6.13 (模型更新原则). 计划、需求、架构和估算均可视为关于世界的模型。获得新证据后, 应根据证据质量与更新成本修正模型。

写成

$$M_t \xrightarrow{O_{t+1}} M_{t+1}.$$

该原则由

$$K + C + \Sigma + G$$

驱动, 并支持滚动计划、持续需求发现、性能校准和基于结果的技术选择。

7 可检验假设与机制评价

为使 P1–P13 形成可积累的理论对象, 定义每条原则的假设记录为

$$\mathcal{H}_i = (\text{前提}, \text{机制}, \text{指标}, \text{方向与边界}).$$

其中, 前提由 S_Q 、 K 、 R 、 C 、 I 、 G 、 E 和 Σ 的具体状态给出; 机制说明中间变量如何进入策略价值; 指标提供跨项目观测接口; 方向与边界给出可检验的条件性预测。完整的 P1–P13 假设记录见附录 C (表 3)。正文仅报告其统一编码方式、机制层分组和评价方法。

7.1 机制层压缩与消融评价

将 P1–P13 组织为四个可比较的机制层: 信息与学习层 $\{P_1, P_5, P_6, P_{13}\}$ 、选择权与恢复层 $\{P_2, P_{10}\}$ 、复杂度与局部化层 $\{P_3, P_4, P_9\}$ 、权威与执行层 $\{P_7, P_8, P_{11}, P_{12}\}$ 。机制层数量记为 M , 经验原则语料中的可解释条目数量记为 N , 定义压缩比为

$$\rho = \frac{N}{M}.$$

对每条原则 P_i 定义消融增量

$$\Delta_i = \text{Score}(\mathcal{H}) - \text{Score}(\mathcal{H} \setminus \{P_i\}),$$

其中 Score 综合预测校准、机制覆盖、增量解释力和反例识别率。 ρ 表示机制层的覆盖效率, Δ_i 表示单条原则对整体解释与预测表现的边际贡献。跨语料比较机制层数量、压缩比和消融增量, 可以给出原则集合的最小化与扩展依据。

8 代表性实践推导

8.1 DRY

设同一业务知识 q 被复制到 n 个位置:

$$q_1, q_2, \dots, q_n.$$

每次规则变化需要同步更新多个副本, 产生维护成本:

$$C_{\text{update}} \propto n.$$

由于 C, 无法保证所有修改在所有副本上同步发生; 由于 I, 副本分叉以后需要付出重新确认权威、协调冲突和恢复一致性的成本; 由于 R, 持续人工同步受到预算约束。

因此:

$$R + C + I + G \rightsquigarrow P7 \rightsquigarrow \text{DRY}.$$

这一推导把 DRY 的核心对象定位为“权威知识”。两个代码片段拥有相似文本, 同时表达独立业务语义时, 可以保持独立演化。

8.2 YAGNI 与避免预言未来

未来需求属于当前未知:

$$K.$$

提前实现未来功能立即消耗资源:

$$R.$$

加入接口、数据结构和依赖会收缩未来可选行动, 并提高恢复到原有设计空间的成本:

$$I.$$

因此, 在需求尚未获得足够证据时:

$$K + R + I + G \rightsquigarrow P2 + P3 \rightsquigarrow \text{延迟未经证实的结构承诺}.$$

8.3 版本控制

代码修改受到 C: 误操作、错误假设、依赖变化和协作冲突均可能发生。若直接覆盖旧状态且缺少恢复证据 z , 行动的 $I_{\epsilon, \delta}$ 会升高; 人工保留历史又持续消耗 R。

因此:

$$C + I + R + G \rightsquigarrow P6 + P10 + P8 \rightsquigarrow \text{版本控制}.$$

版本控制把历史纳入可用证据 z , 同时提供恢复路径并降低历史保存的边际成本, 因此能够系统性降低代码变更的工程不可逆性。

8.4 自动化测试

当前代码正确性受到 K; 未来输入和依赖行为受到 C; 人工回归受到 R。

在 Σ 成立时，测试结果对未来故障具有信息价值：

$$K + C + R + \Sigma + G \rightsquigarrow P1 + P5 + P8 \rightsquigarrow \text{自动化测试.}$$

测试还能把已发现行为规则外部化：

$$P12.$$

因此“每个 Bug 只需被发现一次”可进一步表示为

$$\text{Bug 证据} \rightsquigarrow \text{回归测试} \rightsquigarrow \text{长期保存新获得的不变量知识.}$$

8.5 低耦合

若一个模块变化会迫使大量模块共同修改，则一次变化的知识需求、协调成本和不可控传播范围同时增加。

因此：

$$K + R + C + G \rightsquigarrow P4 \rightsquigarrow \text{低耦合、封装与局部化.}$$

进一步加入接口边界成本，可获得条件化结论：

$$\min (C_{\text{change propagation}} + C_{\text{boundary}}).$$

该表达可以同时评价模块化单体与分布式服务架构。

8.6 显式时间语义

设系统需要表达一个确定时刻和其业务时区语义：

$$x = (t, z).$$

若传输为

$$f(t, z) = t_{\text{local}},$$

多个 (t, z) 可能产生相同字符串。I 表明在缺少额外证据时无法精确恢复原有时间语义；K 表明接收方当前无法确定发送方采用了哪一种隐式上下文。

因此：

$$K + I \rightsquigarrow P5 + P6 \rightsquigarrow \text{显式携带 Instant、offset、zone 或明确的时间语义.}$$

9 案例研究：直接发布与分阶段发布

本节先给出一个静态单阶段基准，再给出同一 Beta-Binomial 证据模型下的有限时域序贯递推。所有数值均为可复算的合成输入，用于展示策略边界、预算约束和模型更新。

9.1 静态单阶段基准

团队准备发布一项会改变第三方客户端交互方式的功能。缺陷概率先验为

$$q \sim \text{Beta}(1, 9).$$

在 40 个集成场景中观察到 3 个缺陷和 37 个成功结果，得到

$$q \mid o \sim \text{Beta}(4, 46), \quad \bar{q} = \mathbb{E}[q \mid o] = 0.08.$$

比较直接发布 a_D 与使用 feature flag 的分阶段发布 a_S 。价值和成本统一换算为工程小时等价成本 (EH)：

变量	直接发布 a_D	分阶段发布 a_S
成功交付价值 V	240	240
实现与发布成本 C	20	35
延迟价值损失 C_{delay}	0	12
缺陷发生后的直接损失 L	600	60
工程不可逆性 $I_{0,0.01}$	80	6

直接发布和分阶段发布的期望效用分别为

$$\mathbb{E}[U_D] = 0.92 \times 240 - 20 - 0.08(600 + 80) = 146.40,$$

$$\mathbb{E}[U_S] = 0.92 \times 240 - 35 - 12 - 0.08(60 + 6) = 168.52.$$

因此在该单阶段参数下， $\mathbb{E}[U_S] - \mathbb{E}[U_D] = 22.12$ 。两项策略使用相同的缺陷概率、价值尺度和恢复成本定义； L 记录直接后果， I 记录恢复到任务等价状态的资源成本，每项代价在价值账本中保持单独归属。

静态效用差的敏感性表达为

$$\mathbb{E}[U_S - U_D] = -27 + 614q.$$

分阶段发布的策略边界为

$$q > \frac{27}{614} \approx 0.0440.$$

前置预算低于 35 EH 时， a_S 不属于当前可行集合； Σ 的稳定性参数变化时，重新估计 L_S ；伦理观测约束变化时，重新登记证据 z 和恢复策略集合。

9.2 有限时域序贯递推

令一次试点观察 $X \in \{0, \dots, m\}$ 个缺陷，满足

$$X \mid q \sim \text{Binomial}(m, q), \quad q \mid o \sim \text{Beta}(\alpha, \beta).$$

观察 $X = x$ 后,

$$q \mid o, x \sim \text{Beta}(\alpha + x, \beta + m - x), \quad \bar{q}_x = \frac{\alpha + x}{\alpha + \beta + m}.$$

试点后的决策集合定义为 $\mathcal{B} = \{e, r, o\}$, 分别表示扩大、回滚和继续观察。给定剩余试点轮数 n 、预算 b 、后验参数 (α, β) 和活动状态 $z \in \{0, 1\}$, 定义

$$V_n(\alpha, \beta, b, z) = \max_{a \in \mathcal{B}_n} \mathbb{E}[U(a) \mid \alpha, \beta, b, z]. \quad (9.1)$$

终止扩大和回滚的价值为

$$V_e(\alpha, \beta) = (1 - \bar{q})V - C_e - \bar{q}(L_e + I_e), \quad V_r = -C_r. \quad (9.2)$$

继续观察的价值为

$$V_o = -C_o - C_{\text{delay}} + \sum_{x=0}^m \Pr(x \mid \alpha, \beta) [-\ell x + V_{n-1}(\alpha + x, \beta + m - x, b - C_o, 1)], \quad (9.3)$$

其中

$$\Pr(x \mid \alpha, \beta) = \binom{m}{x} \frac{B(\alpha + x, \beta + m - x)}{B(\alpha, \beta)}. \quad (9.4)$$

当 $n = 0$ 时, $\mathcal{B}_n$ 只包含当前可行的终止行动; 当 $z = 0$ 时, 保留停止行动 $V_{\text{stop}} = 0$ 。最优策略为

$$a_n^*(\alpha, \beta, b, z) = \arg \max_{a \in \mathcal{B}_n} \mathbb{E}[U(a) \mid \alpha, \beta, b, z].$$

该递推在每次观察后更新后验, 并在预算、恢复能力和伦理边界确定的条件下给出扩大、回滚、继续观察或停止的策略。

9.3 可复现的数值求解

附带脚本 `examples/sequential_release.py` 使用合成参数

$$(\alpha, \beta, m, n) = (4, 46, 10, 2), \quad (V, C_e, L_e, I_e, C_r) = (240, 20, 2400, 80, 6),$$

$$(C_o, C_{\text{delay}}, \ell, B_0, B_{\text{rec}}) = (2, 1, 3, 30, 100).$$

观测信号采用交换性 Beta-Binomial 模型; 试点失败的直接代价为 ℓx , 恢复预算单独登记。精确后向递推得到

$$V_2(4, 46, 30, 0) = 12.314852, \quad V_1(4, 46, 30, 0) = 9.393790,$$

而根节点三个可行行动的价值为

$$V_{\text{stop}} = 0, \quad V_e = 2.400000, \quad V_o = 12.314852.$$

因此根节点选择继续观察。第一轮观察后的策略为： $x = 0$ 时扩大， $x = 1$ 时继续观察， $x \geq 2$ 时回滚；相应的预测概率为

$$\Pr(x = 0) = 0.465533, \quad \Pr(x = 1) = 0.338569, \quad \Pr(x \geq 2) = 0.195899.$$

脚本逐项检查预测概率归一化、后验均值鞅性质、独立枚举与递推值一致性，并输出完整的后验策略表。该计算展示序贯反馈相对于单轮观察的选择权价值；输入参数、策略集合和预算可以直接替换以复算其他发布情境。

10 讨论

10.1 原则冲突与条件化推导

KRCIGE 的一个重要用途是处理经验原则之间的张力。此时目标是寻找多个约束共同作用下的最优区域。

10.1.1 DRY 与抽象时机

DRY 主要由 $P7$ 支持；抽象时机主要由 $P2$ 支持。

当多个实现仍缺少足够证据证明它们承载同一稳定知识时， K 较强，选择权价值较高。随着重复实例增加、共同变化模式变得清晰， $P7$ 的收益逐渐上升。

由此得到：

先观察共同变化，再抽取共同权威知识。

10.1.2 Fail Fast 与 Graceful Degradation

靠近错误源时，继续传播错误状态会损失诊断信息，因此 $P5 + P6$ 支持快速暴露。

靠近用户价值边界时，服务整体失效可能扩大损失，因此 $P10 + P11$ 支持隔离、降级和恢复。

可形成分层规则：

局部尽早暴露错误，系统边界限制故障传播。

10.1.3 小步迭代与原子不变量

$P2$ 支持较小承诺； $P10$ 支持可恢复步骤。若一个不变量要求若干变化共同生效，则最小安全步骤由不变量边界决定。

因此得到：

选择能够独立保持关键不变量的最小可恢复步骤。

10.1.4 配置化与状态空间

外部配置通过 $P2$ 保留策略选择权，同时每增加一个运行时参数都会扩大 $P9$ 所描述的状态空间。

因此:

高变化频率策略适合显式配置; 高稳定度不变量适合固化在类型、schema 与代码约束中。

10.1.5 Postel 鲁棒性原则

协议接收方容忍更多输入可以吸收 C 带来的外部实现差异; 同时, 过度容忍会降低发送方获得错误反馈的机会, 并允许歧义长期存在。RFC 9413 对鲁棒性原则的长期互操作性影响进行了系统讨论 [12]。

KRCIGE 可将这一张力表示为

$$C \text{ 与 } K+I$$

之间的权衡。

对于缺少协调能力的旧生态, 兼容容忍具有较高价值; 对于可主动维护的新协议, 严格验证能够形成更高质量的反馈与更小的协议状态空间。

10.2 进一步原则

以下原则超出 *The Pragmatic Programmer* Tip 表中的直接表达, 用于展示框架的生成性。

10.2.1 证据半衰期原则

有些故障证据会迅速消失。例如一次请求的原始 payload、线程调度、第三方响应和内存现场。设时刻 t 可用于恢复的证据为 z_t 。在没有新增证据且证据集合随时间收缩时, 通常有

$$z_{t+1} \subseteq z_t \rightsquigarrow I_{\varepsilon, \delta}(a \mid h_t, z_{t+1}) \geq I_{\varepsilon, \delta}(a \mid h_t, z_t).$$

定义证据价值

$$V_e(t)$$

随时间下降。若

$$\frac{dV_e}{dt} \ll 0,$$

则应靠近产生点采集证据。

由此得到:

越难重现的证据, 越应靠近事件源自动捕获。

10.2.2 保证-假设对偶

任何强保证都建立在一组系统假设上。将保证记为 G_u , 假设集合记为

$$\mathcal{H} = \{H_1, \dots, H_n\}.$$

则工程评审应同时记录:

$$G_u \leftrightarrow \mathcal{H}.$$

FLP 结果提供了这一思想的经典极端案例: 异步性和故障模型直接约束可实现的共识保证 [7]。

10.2.3 副本权威原则

允许数据拥有多个副本，同时要求：

replication $\rightsquigarrow$ authority 或 reconciliation semantics.

cache、read replica、event projection 和多地域副本均可在这一原则下分析。

11 理论边界与要素区分

下表用反例区分各要素，避免把不同约束合并为笼统的“困难”。

区分	反例
K / R	不可观测隐藏位在资源充足时仍然未知；完全可观测的有限状态机仍消耗时间、内存和能量。
K / C	主体可以知道天气而无法改变天气，也可以不知道寄存器旧值而直接控制其下一状态。
I / K	已知删除生产数据的后果仍可能无法恢复；不了解新功能效果时，沙箱和 feature flag 可以保持低 I。
I / R	I 给出达到恢复要求的最低负担，R 给出主体可支配的预算；相同恢复任务对不同预算主体具有不同可行性。
I / C	C 限定可执行的恢复策略集合 Π_E ；第三方接口、人员行为和外部传播会改变 I。

前文的多对一变换进一步给出信息擦除的结构性案例。

KRCI 描述约束，G 指定优化方向，E 保留不可交换的伦理、安全和责任边界。涉及心理、团队、组织或具体工具效果的路径标记为辅助经验前提 Ψ 。该标记只声明路径依赖外部证据；每项使用都需要具体主张、适用范围和相应实证支持，含 Ψ 的路径不计作 KRCIGE 核心独立解释。

12 实证检验设计

KRCIGE 的理论评价由解释覆盖、方向预测、边界识别和相对于基线模型的增量解释力共同构成。构造语料用于展示概念覆盖，独立语料用于评估预测与机制；两类材料分别登记并保持分析边界。

12.1 内部留出核验

The Pragmatic Programmer 100 Tips 是构造语料，其完整映射位于附录；Twelve-Factor App 的 12 个因素构成第一组内部留出材料 [11]，没有用于新增基础要素或修改 P1–P13。独立评估材料由软件事故报告、架构决策记录、发布与回滚日志、数据迁移案例和设计实验组成，并覆盖正向、反向与边界案例。

本轮验证采用以下顺序：

1. 冻结 KRCIGE 定义、P1–P13 与推导符号；

2. 只依据 Twelve-Factor 官方索引中的因素名称和一句式副标题，记录主要约束、预测路径及方向条件；
3. 再读取每个因素的官方详细说明，检查预测机制与方向是否出现；
4. 核验期间不新增中间原则，也不回写预测路径。

分类规则预先固定为：若材料清楚支持冻结预测的主要机制和方向，记为“直接支持”；若支持主要机制，但方向条件或主要路径只有部分吻合，记为“部分支持”；若材料呈现新的机制接口，记录为“扩展接口”；若材料提供边界或反向案例，记录为“边界案例”。该分类同时记录机制、指标、方向和适用条件。完整的 12 项逐条核验表移至附录 B。

分类结果为 10 项直接支持、2 项部分支持，未出现“未支持”或“方向相反”，也没有触发新基础约束。该计数仅描述本次分类分布，不换算为命中率；Port binding 与 Admin processes 保留为部分支持，以呈现冻结预测与官方论述的差异。

证据等级限于内部前瞻性核验：单一评审者可能接触过相关思想，材料来自同一文档家族，多标签路径也具有自由度。下一步需要公开冻结预测、盲态独立编码、评审一致性统计和第二类领域材料。

12.2 独立评估协议

独立评估以项目为分析单位，以冻结的 $\mathcal{H}_i$ 为预测注册表。每个项目记录任务 Q 、主体 s 、时间窗 τ 、决策前观测、策略选择、资源预算、控制扰动、 $I_{\varepsilon,\delta}$ 代理和结果指标。评估材料分为四类：事故与恢复记录、架构决策记录、发布与回滚日志、以及包含正向和反向案例的设计实验。

编码流程定义为：

冻结预测 $\rightarrow$ 材料抽样 $\rightarrow$ 盲态独立编码 $\rightarrow$ 一致性统计 $\rightarrow$ 结果揭盲与更新。

至少两名编码者依据同一份编码手册记录机制、指标、方向和边界；分类一致性使用 Cohen's κ 或 Krippendorff's α 表示。分析同时设置三类基线：仅使用经验规则的基线、仅使用一般风险变量的基线、以及包含 KRCIGE 变量的完整模型。

评估指标定义为：

$$\begin{aligned} \text{预测校准} &= 1 - \text{BrierScore}, \\ \text{解释覆盖} &= \frac{\text{得到机制编码的案例数}}{\text{案例总数}}, \\ \text{增量解释力} &= \Delta R^2 \text{ 或 } \Delta \text{LogLoss}, \\ \text{反例识别率} &= \frac{\text{正确标记适用边界的案例数}}{\text{边界案例总数}}. \end{aligned}$$

独立数据集上的预测校准、机制覆盖、增量解释力和反例识别率共同构成 KRCIGE 的实证评价结果。结果、编码手册、数据字典和分析脚本应以可复现格式公开。

12.3 可反驳条件

以下结果会削弱 KRCIGE：

1. 大量核心工程原则需要持续引入彼此无关的新基础约束；
2. 同一个基础条件经常同时推出方向相反且无法由成本、目标或层级区分的策略；
3. KRCIGE 对原则适用条件的预测与实证结果长期系统性相反；
4. 中间原则数量持续接近经验原则数量，压缩优势消失；

5. Σ 、 Ψ 等辅助条件不断扩张, 使任意观察都能被事后吸收。

这些标准能够约束框架复杂度, 降低循环解释的风险。

12.4 可量化研究方向

跨项目研究应使用可观测代理、区间或后验分布表示 k_Q 、 λ_j 、 γ_Q 和 $I_{\epsilon,\delta}$, 并公开测量误差与模型假设。恢复演练可以提供已知恢复策略的成本和成功率; 信息丢失证明可以提供恢复能力的结构性下界。可检验的方向性预测包括:

$$k_Q \uparrow \rightsquigarrow \text{在其他条件相近时, 小步实验和可逆性投入上升,} \quad (12.1)$$

$$I_{\epsilon,\delta} \uparrow \rightsquigarrow \text{在防护有效时, review、备份、授权与审计价值上升,} \quad (12.2)$$

$$\gamma_Q \uparrow \rightsquigarrow \text{冗余、隔离、超时和恢复机制投入上升,} \quad (12.3)$$

$$\lambda_j \uparrow \rightsquigarrow \text{围绕资源 } j \text{ 的复杂度预算趋紧, 自动化与资源替代价值变化.} \quad (12.4)$$

项目历史、事故报告、设计实验和组织对照研究可以用于检验这些关系。

13 应用范围与使用流程

13.1 适用范围

KRCIGE 当前覆盖程序与 API、数据表示、架构边界、测试观测、并发与分布式系统、发布恢复、部分安全工程和需求迭代。团队心理、组织政治、激励机制、职业成长与宏观社会伦理可以通过多主体模型和相应领域理论接入; Ψ 与 E 提供显式接口。

13.2 使用清单

应用框架时依次回答:

1. **情境**: 明确任务 Q 、主体 s 和评估时间窗 τ , 记录行动集合、状态转移和目标状态集合。
2. **K**: 当前有哪些关键未知量?
3. **R**: 各类资源预算、消耗和影子价格 λ_j 如何确定?
4. **C**: 控制权限、扰动集合和控制缺口 γ_Q 如何确定?
5. **I**: 行动后恢复相关信息、状态和未来选择需要多大成本, 能够达到什么质量与可靠性?
6. **G**: 当前真正优化的价值是什么?
7. **E**: 哪些行动受到伦理、安全与责任边界约束?
8. Σ : 哪些反馈在参考环境中具有稳定的正决策价值?
9. **验证**: 为相关原则登记指标、方向、反转条件和数据来源, 并设置独立评估与基线模型。

随后再选择反馈、模块化、自动化、冗余、测试、日志、配置或版本控制等技术。

14 结论

KRCIGE 将软件工程实践组织为“描述性约束、规范方向与适用条件”到条件性策略的序贯决策理论。统一情境模型连接了决策相关认识缺口 k_Q 、资源影子价格 λ_j 、控制缺口 γ_Q 、工程不可逆性 $I_{\epsilon,\delta}$ 和策略价值。其主要理论增量是: 以恢复成本定义工程不可逆性, 以可检验假设记录连接十三条中间原则, 并以序贯发布递推展示反馈、选择权、恢复和模型更新的共同作用。

数值案例给出静态阈值和序贯策略的计算接口；100 Tips 映射与 Twelve-Factor 内部留出核
验提供探索性覆盖材料；独立评估协议提供跨项目、盲态编码、基线比较和结果复现的研究路
径。后续工作可以围绕 I 的增量解释力、 k_Q 与 γ_Q 的测量校准、原则消融和多主体组织模型展
开。

A 对《The Pragmatic Programmer》100 条 Tips 的映射

The Pragmatic Programmer 20 周年版公开列出 100 条 Tips [9, 10]。本节将其作为经验语
料，对 KRCIGE 的压缩能力进行初步检验。这里的“推导”表示解释路径；其中涉及心理学、组
织行为和职业文化的条目，会额外标记辅助经验前提 Ψ 。

为便于阅读，使用以下缩写：

P_1 = 反馈价值, P_2 = 选择权, P_3 = 复杂度预算,
 P_4 = 局部化, P_5 = 显式假设, P_6 = 信息保留,
 P_7 = 权威来源, P_8 = 自动化, P_9 = 状态空间,
 P_{10} = 可恢复性, P_{11} = 安全约束, P_{12} = 知识外部化,
 P_{13} = 模型更新.

表 1: 100 条 Tips 与 KRCIGE 的初步推导路径

#	Tip (中文概括)	路径
1	认真打磨自己的专业能力	$K + R + G + \Psi \rightsquigarrow P_{12} \rightsquigarrow$ 持续提高能力与知识质量。
2	思考你的工作	$K + \Sigma + G \rightsquigarrow P_1 + P_{13} \rightsquigarrow$ 持续反思并更新工作模型。
3	你拥有主动权	$C + G \rightsquigarrow$ 识别可控行动子集 $\rightsquigarrow P_{13}$ 。
4	提供可选方案，别拿蹩脚借口搪塞	$C + K + G \rightsquigarrow P_2 \rightsquigarrow$ 在可控范围提供选择与代价。
5	不要容忍已经存在的缺陷、糟糕设计 和错误决定	$C + I + R + G \rightsquigarrow P_{10} + P_3 \rightsquigarrow$ 趁上下文与修复成本仍低 时处理退化。
6	主动推动有益的改变发生	$C + G + \Psi \rightsquigarrow$ 从可控局部形成反馈与扩散。
7	持续关注系统的整体目标和影响	$K + R + G \rightsquigarrow P_4 \rightsquigarrow$ 局部决策同时检查系统级目标。
8	把质量作为明确的需求来讨论	$K + R + G \rightsquigarrow P_3 + P_{13} \rightsquigarrow$ 将质量、成本与价值显式建模。
9	持续投资你的知识和技能	$K + C + R + G \rightsquigarrow P_{12} + P_{13}$ 。
10	批判性地分析你读到和听到的内容	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
11	像编写代码一样清晰、简洁、可维 护地使用自然语言	$K + I \rightsquigarrow P_5 + P_{12} \rightsquigarrow$ 语言同样承载可丢失的工程语义。
12	你说什么重要，你怎么说同样重要	$K + I \rightsquigarrow P_6 + P_{12}$ 。
13	把文档直接构建到系统和工作流中	$K + I + R + C \rightsquigarrow P_{12} + P_8$ 。
14	好设计比坏设计更容易修改	$K + I + R + C + G \rightsquigarrow P_2 + P_4$ 。
15	DRY——避免重复表达同一份知识	$R + C + I + G \rightsquigarrow P_7$ 。
16	让复用变得容易	$R + G \rightsquigarrow P_3 + P_7 + P_4$ 。
17	消除无关事物之间的影响	$K + R + C + G \rightsquigarrow P_4$ 。
18	尽量保留调整和撤销决策的空间	$K + I + G \rightsquigarrow P_2$ 。
19	根据实际问题 and 证据选择技术，避 免盲目追逐潮流	$K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}$ 。

#	Tip (中文概括)	路径
20	尽早构建可运行的局部方案, 用实际结果验证方向	$K + C + \Sigma + R + G \rightsquigarrow P_1 + P_2。$
21	用原型验证理解、需求和技术方案	$K + \Sigma + R + I + G \rightsquigarrow P_1 + P_2。$
22	让程序贴近真实业务和问题背景	$K + I + R \rightsquigarrow P_5 + P_{12} + P_3。$
23	用估算提前暴露不确定性和风险	$K + R + G \rightsquigarrow P_{13} + P_3。$
24	根据代码验证结果持续调整进度计划	$K + C + \Sigma + G \rightsquigarrow P_{13}。$
25	用纯文本保存知识	$I + K + G \rightsquigarrow P_6 + P_{12} + P_2, \text{ 具体介质选择依赖 } \Psi。$
26	用命令 Shell 组合和自动化工作流程	$R + G + \Psi \rightsquigarrow P_8 + P_3。$
27	熟练掌握编辑器, 让编辑操作高效而自然	$R + G + \Psi \rightsquigarrow P_3。$
28	始终使用版本控制	$C + I + R + G \rightsquigarrow P_6 + P_{10} + P_8。$
29	修复问题, 别追究责任	$R + K + G + \Psi \rightsquigarrow P_1 + P_{12}。$
30	保持冷静	$R + G + \Psi \rightsquigarrow \text{ 保护有限认知资源。}$
31	先写一个会失败的测试, 再修复代码	$K + \Sigma + G \rightsquigarrow P_1 + P_5。$
32	仔细读懂错误消息	$K + I + G \rightsquigarrow P_6 + P_1。$
33	先检查查询、数据和假设, 别急着认为 <code>select</code> 出错	$K + \Sigma + \Psi \rightsquigarrow P_1, \text{ 结合成熟组件的经验先验。}$
34	别想当然, 用证据验证	$K + \Sigma + G \rightsquigarrow P_1。$
35	学一门文本处理语言	$R + G + \Psi \rightsquigarrow P_8 + P_3。$
36	承认软件不可能完美, 持续改进它	$K + R + C + G \rightsquigarrow P_3。$
37	用明确的前置条件、后置条件和不变量设计代码	$K + C + I \rightsquigarrow P_5。$
38	尽早发现严重错误并停止执行, 避免继续产生错误结果	$C + I + G \rightsquigarrow P_5 + P_6 + P_{10}。$
39	用断言检查不变量, 尽早捕获不应发生的状态	$K + C + \Sigma \rightsquigarrow P_5 + P_1。$
40	完成已经开始的工作, 或明确结束它的方式	$R + I + G + \Psi \rightsquigarrow P_3 + P_{12}。$
41	把变化和影响限制在局部范围内	$K + R + C + G \rightsquigarrow P_4。$
42	每次只做小幅、可验证的改动	$K + C + I + \Sigma + G \rightsquigarrow P_1 + P_2 + P_{10}。$
43	不要假设未来需求和技术会怎样, 依据当前证据做可调整的决策	$K + I + R + G \rightsquigarrow P_2 + P_3。$
44	减少模块之间的依赖, 让代码更容易修改	$K + R + C + G \rightsquigarrow P_4。$
45	让对象直接执行操作, 通过行为接口协作, 减少对内部状态的询问	$K + R + C \rightsquigarrow P_4 + P_5。$
46	避免让方法调用形成过长的链条	$K + R + C \rightsquigarrow P_4。$
47	避免全局数据	$K + C + R \rightsquigarrow P_9 + P_4。$
48	通过 API 管理需要全局共享的重要资源	$K + C + I \rightsquigarrow P_5 + P_4 + P_9。$

#	Tip (中文概括)	路径
49	编程要写代码, 但程序最终处理的是数据	$K + I + G \rightsquigarrow P_5 + P_6$ 。
50	减少状态的集中持有, 让状态沿数据流传递	$K + C + I \rightsquigarrow P_5 + P_9$ 。
51	警惕继承带来的耦合和维护成本	$R + K + C + G \rightsquigarrow P_3 + P_4$ 。
52	优先用接口表达不同实现之间的多态关系	$K + I + G \rightsquigarrow P_2 + P_4$ 。
53	优先通过服务委托和组合复用功能, 减少对继承的依赖	$K + I + R + G \rightsquigarrow P_2 + P_4 + P_3$ 。
54	用 mixin 共享可复用功能	$R + G + \Psi \rightsquigarrow P_7 + P_4$ 。
55	让应用通过外部配置适配不同环境	$K + C + I + G \rightsquigarrow P_2 + P_5 + P_9$ 。
56	分析工作流, 找出可以并发执行的部分	$R + K + \Sigma + G \rightsquigarrow P_1 + P_9$ 。
57	尽量避免共享可变状态, 因为它容易导致不一致和并发错误	$K + R + C \rightsquigarrow P_9$ 。
58	随机失败往往是并发问题	$C + K + \Psi \rightsquigarrow P_1 + P_9$ 。
59	使用 Actor 模型, 在无共享状态的前提下实现并发	$K + R + C \rightsquigarrow P_9 + P_4$ 。
60	用共享的信息空间协调工作流	$K + C + I + \Psi \rightsquigarrow P_5 + P_9 + P_4$ 。
61	识别并管理面对风险和批评时的本能防御反应	$K + R + \Psi \rightsquigarrow P_1$, 将直觉作为待验证信号。
62	不要依赖碰巧成立的行为、顺序或环境来编程	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
63	估算算法随输入规模增长的资源消耗	$R + G \rightsquigarrow P_3$ 。
64	用实际测试检验对算法性能的估算	$K + \Sigma + G \rightsquigarrow P_1 + P_{13}$ 。
65	尽早重构, 持续重构	$K + R + C + I + G \rightsquigarrow P_2 + P_3 + P_4 + P_{12}$ 。
66	测试用于验证行为、保护回归, 并发现 Bug	$K + I + \Sigma \rightsquigarrow P_5 + P_{12}$ 。
67	先用测试验证代码的使用方式和接口	$K + R + G \rightsquigarrow P_1 + P_4$ 。
68	先构建贯穿完整流程的可运行方案, 再逐步扩展	$K + C + \Sigma + G \rightsquigarrow P_1$ 。
69	为可测试性而设计	$K + C + \Sigma + G \rightsquigarrow P_1 + P_4 + P_5$ 。
70	测试你的软件, 避免把缺陷暴露给用户	$K + C + \Sigma + G \rightsquigarrow P_1$ 。
71	用基于属性的测试验证对系统行为的假设	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
72	保持简单, 减少可被攻击的入口	$R + C + I + E \rightsquigarrow P_3 + P_{11}$ 。
73	快速应用安全补丁	$C + I + E + G \rightsquigarrow P_{10} + P_{11}$ 。
74	选择清晰准确的名称, 需要时及时重命名	$K + I + G \rightsquigarrow P_5 + P_{12}$ 。
75	承认需求一开始通常并不明确	K 的需求领域实例。
76	帮助用户把模糊需求说清楚	$K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}$ 。

#	Tip (中文概括)	路径
77	通过反复收集反馈和调整实现来逐步明确需求	$K + C + \Sigma + G \rightsquigarrow P_1 + P_{13}。$
78	和用户一起工作，站在用户角度思考	$K + \Sigma + G \rightsquigarrow P_1 + P_{12}。$
79	把变化频繁的策略作为可配置的元数据管理	$K + C + I + G \rightsquigarrow P_2 + P_5 + P_9。$
80	建立并使用统一的项目术语表	$K + I + R \rightsquigarrow P_5 + P_{12}。$
81	先识别问题和约束边界，再寻找突破方式	$K + \Sigma + G \rightsquigarrow P_1 + P_{13}。$
82	通过结对编程、代码评审或团队协作来编写代码	$K + R + I + \Psi \rightsquigarrow P_{12}。$
83	把敏捷落实为持续反馈、快速交付和持续改进的做事方式	$K + C + \Sigma + I + G \rightsquigarrow P_1 + P_2 + P_{13}。$
84	保持团队规模小而稳定	$R + I + \Psi \rightsquigarrow P_3 + P_{12}。$
85	把重要的改进和学习安排进日程，才能落实	$R + G + \Psi \rightsquigarrow$ 显式资源配置。
86	组建能够独立完成端到端工作的团队	$K + R + C + G + \Psi \rightsquigarrow P_4 + P_{12}。$
87	根据实际效果做事，别追逐时髦	$K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}。$
88	在用户真正需要时交付价值	$R + C + G \rightsquigarrow P_{13}$ ，价值时点由 G 给出。
89	让版本控制触发构建、测试和发布	$C + I + R + G \rightsquigarrow P_6 + P_8 + P_{10}。$
90	尽早测试、频繁测试，并自动执行测试	$K + C + R + \Sigma + G \rightsquigarrow P_1 + P_5 + P_8。$
91	所有测试跑完，编码才算完成	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ ，完成定义来自 G 。
92	用故意制造错误的测试程序检验测试	$K + \Sigma + G \rightsquigarrow P_1 + P_5。$
93	以状态覆盖率检验测试，别只看代码覆盖率	$K + I + \Sigma \rightsquigarrow P_5 + P_9。$
94	每个已发现的 Bug 都要用自动化测试固定下来，避免再次出现	$K + I + R + G \rightsquigarrow P_6 + P_{12} + P_5。$
95	把手工流程自动化	$R + C + G \rightsquigarrow P_8。$
96	让用户获得满意和惊喜，交付可用价值	$K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}。$
97	为自己的成果署名并承担责任	$G + E + \Psi \rightsquigarrow$ 责任与可追溯性；KRCI 可解释其工程收益。
98	优先确保软件不造成伤害	$E \rightsquigarrow P_{11}$ ； $C + I$ 提高高损害行动的谨慎级别。
99	不要为恶意使用者提供便利	$E + C + I \rightsquigarrow P_{11}。$
100	分享成果，庆祝进展，持续改进和建设	$K + I + R + G + \Psi \rightsquigarrow P_{12} + P_{13}$ ；庆祝属于组织文化前提。

该表展示 P1–P13 在构造语料中的复用情况；它用于检查压缩性，不承担独立验证功能。

B Twelve-Factor App 留出集

本附录保留内部留出核验的完整逐项记录。核验材料只用于评估冻结预测，不用于修改 KRCIGE 的基础要素或 P1–P13。

表 2: Twelve-Factor App 留出集预测与核验结果

#	留出因素	冻结预测	官方说明核验与分类
1	Codebase	$P7 + P6$; 部署分叉增多时, 权威来源与版本历史价值上升。	官方要求一个代码库对应一个应用、由版本控制追踪并支持多个部署, 覆盖权威来源和历史保留。分类: 直接支持。
2	Dependencies	$P5 + P7 + P10$; 执行环境差异越大, 显式声明和隔离的价值越高。	官方要求完整声明、隔离依赖, 并以未来机器可能缺包或版本不兼容解释其价值。分类: 直接支持。
3	Config	$P2 + P5 + P9$; 部署间变化越频繁, 配置与代码分离的价值越高。	官方把配置定义为部署间变化项, 要求独立、正交管理, 并讨论组合爆炸。分类: 直接支持。
4	Backing services	$P4 + P5 + P10$; 外部服务替换和故障概率上升时, 附着式边界价值上升。	官方要求以资源句柄连接服务, 并给出数据库故障后从备份恢复、重新挂接且无需改代码的例子。分类: 直接支持。
5	Build, release, run	$P4 + P6 + P10$; 发布恢复要求越高, 阶段分离和不可变发布账本越有价值。	官方严格区分三个阶段, 要求发布具有唯一 ID、保持追加式不可变, 并明确支持快速回滚。分类: 直接支持。
6	Processes	$P9 + P4 + P10$; 重启和调度越不可控, 无共享、无状态进程的价值越高。	官方要求进程无状态且无共享, 并以重启、迁移和不同进程处理后续请求说明本地状态不可靠。分类: 直接支持。
7	Port binding	$P4 + P5$; 运行容器差异增大时, 自包含服务和显式端口契约价值上升。	官方支持自包含服务和端口契约, 机制吻合; 详细说明没有直接比较容器差异变化时的策略强度。分类: 部分支持。
8	Concurrency	$P9 + P3 + P13$; 负载与工作类型增加时, 按进程类型分解和横向扩展价值上升。	官方以进程类型表达工作差异, 以无共享和水平分区解释可靠扩展, 并讨论纵向扩展限制。分类: 直接支持。
9	Disposability	$P10 + P4$; 终止和迁移越频繁, 快速启动、优雅退出与幂等价值上升。	官方直接以弹性扩缩、部署、硬件突然失效和任务重入说明快速启动、优雅退出与幂等。分类: 直接支持。
10	Dev/prod parity	$P1 + P5 + P13$; 环境差异越大, 生产特有错误和反馈延迟越高。	官方列出时间、人员和工具三类差距, 并说明后端服务差异会使测试通过的代码在生产失败。分类: 直接支持。
11	Logs	$P1 + P6 + P12$; 运行时不确定性越高, 事件证据的诊断与历史价值越高。	官方把日志定义为事件流, 并明确用于历史检索、趋势分析和主动告警。分类: 直接支持。
12	Admin processes	$P4 + P8 + P10$; 一次性高风险任务应与常驻流程分离并沿用可恢复发布路径。	官方支持一次性进程及相同代码、配置和依赖, 机制部分吻合; 其主要论证更集中于环境一致性和同步, 未直接给出不可逆性强度关系。分类: 部分支持。

分类结果为 10 项直接支持、2 项部分支持; 该计数仅描述本次分类分布, 不换算为命中率。

C P1–P13 的可检验假设记录

本附录给出正文定义的假设记录 $\mathcal{H}_i$ 的完整版本，供独立编码、消融分析和跨项目比较使用。

表 3: P1–P13 的可检验假设记录

原则	机制	主要指标	方向与适用边界
P1	决策相关观测提升策略价值	NVOI、反馈后的决策变更率、预测误差下降	k_Q 上升且 Σ 稳定时，正净价值观测的投入和决策质量上升；观测成本进入同一比较。
P2	延迟承诺保留未来策略空间	可逆步骤比例、等待期权价值、承诺后的策略数量	k_Q 或 $I_{\epsilon,\delta}$ 上升时，保留选择权的价值上升；延迟价值损失进入策略比较。
P3	复杂度消耗实现、理解、运行和变更资源	四类复杂度成本、资源影子价格 λ_j 、变更前置时间	λ_j 上升时，降低对应复杂度和采用资源替代的边际价值上升；新增复杂度由其交付价值覆盖。
P4	边界降低变化传播和协调范围	变化传播半径、跨团队协调次数、接口边界成本	γ_Q 上升或扰动相关性增强时，局部化收益上升；边界成本纳入总成本。
P5	显式假设提升跨边界验证能力	契约覆盖率、schema 违规率、假设相关缺陷率	跨模块、团队和时间边界的假设密度上升时，显式契约与可执行检查的价值上升。
P6	证据保留降低未来诊断和恢复负担	诊断完整度、恢复成功率、证据检索时间	当 $pL > S + Q$ 时，结构化日志、原始事件和版本历史的保存价值为正。
P7	权威来源或协调语义保持事实一致性	副本冲突率、冲突解决时间、权威解析成功率	副本数量、并发更新和 C 上升时，权威标记或协调语义的价值上升。
P8	自动化以固定成本换取重复执行成本下降	盈亏平衡次数 n^* 、单位执行成本、自动化失败率	当 $F + nC_a + R_a < nC_h + R_h$ 时，自动化具有正期望收益。
P9	状态空间控制降低推理、测试和协调负担	可达状态数、并发交错数、配置组合数、状态覆盖率	共享状态和运行时选项增加时，状态空间约束与明确协调语义的价值上升。
P10	恢复路径降低高不可逆行动的后损失	$I_{\epsilon,\delta}$ 、回滚成功率、MTTR、恢复演练成本	$I_{\epsilon,\delta}$ 和 p_{error} 上升时，防护、备份、幂等和分阶段发布的价值上升。
P11	暴露面和爆炸半径决定安全损失上界	攻击面规模、最小权限覆盖率、单次失效损失上界	γ_Q 、 $I_{\epsilon,\delta}$ 或潜在伤害上升时，隔离、最小权限和数据最小化的价值上升。
P12	知识外部化提升跨主体持续协作能力	bus factor、知识检索时间、文档与测试覆盖率	任务持续时间和主体协作数量上升时，持久化工程知识的价值上升。
P13	新证据更新计划、需求、架构和估算模型	校准误差、后验预测误差、模型更新延迟	证据质量和 Σ 上升时，及时更新模型的预期价值上升；更新成本进入比较。

D 符号表

符号	含义
Q, s, τ	工程任务、主体与有限评估时间窗
S_Q	工程情境及其状态、观测、行动、预算与伦理边界
K	Knowledge bounded: 认识有限
R	Resources bounded: 资源有限
C	Control bounded: 控制有限
I	Irreversibility: 工程不可逆性
k_Q	决策相关认识缺口, 即完整信息的期望决策价值
λ_j	资源 j 的预算边际价值或影子价格
γ_Q	给定扰动集合下的控制缺口
$I_{\varepsilon, \delta}$	达到任务等价恢复要求的最低预期成本
NVOI	观测扣除获取成本后的净决策价值
G	Goal: 工程目标
E	Ethics: 伦理边界
Σ	可学习结构条件
Ψ	心理学、组织行为与具体工具经验等辅助实证前提

参考文献

- [1] Herbert A. Simon. “Rational Decision-Making in Business Organizations.” Nobel Memorial Lecture, 1978. <https://www.nobelprize.org/prizes/economic-sciences/1978/simon/lecture/>
- [2] Ronald A. Howard. “Information Value Theory.” *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966. DOI: <https://doi.org/10.1109/TSSC.1966.300074>
- [3] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956. <https://ashby.info/Ashby-Introduction-to-Cybernetics.pdf>
- [4] Rolf Landauer. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development*, 5(3):183–191, 1961. DOI: <https://doi.org/10.1147/rd.53.0183>
- [5] David L. Parnas. “On the Criteria To Be Used in Decomposing Systems into Modules.” *Communications of the ACM*, 15(12):1053–1058, 1972. DOI: <https://doi.org/10.1145/361598.361623>
- [6] Harold Abelson, Gerald Jay Sussman, with Julie Sussman. *Structure and Interpretation of Computer Programs*, 2nd ed. MIT Press, 1996. <https://mitpress.mit.edu/9780262510875/structure-and-interpretation-of-computer-programs/>

- [7] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM*, 32(2):374–382, 1985. <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
- [8] David H. Wolpert and William G. Macready. “No Free Lunch Theorems for Optimization.” *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. DOI: <https://doi.org/10.1109/4235.585893>
- [9] David Thomas and Andrew Hunt. *The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition*. Addison-Wesley, 2019. ISBN 9780135957059. <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>
- [10] The Pragmatic Bookshelf. “Pragmatic Programmer Tips: 100 Tips from the 20th Anniversary Edition.” <https://pragprog.com/tips/>
- [11] Adam Wiggins. *The Twelve-Factor App*. Last updated 2017. <https://12factor.net/>
- [12] Martin Thomson and David Schinazi. “Maintaining Robust Protocols.” RFC 9413, Internet Architecture Board, June 2023. DOI: <https://doi.org/10.17487/RFC9413>
- [13] Avinash K. Dixit and Robert S. Pindyck. *Investment under Uncertainty*. Princeton University Press, 1994.
- [14] Sigurd Skogestad and Ian Postlethwaite. *Multivariable Feedback Control: Analysis and Design*, 2nd ed. Wiley, 2005.
- [15] Erik Hollnagel, David D. Woods, and Nancy Leveson, editors. *Resilience Engineering: Concepts and Precepts*. Ashgate, 2006.